

Dhuno

Why we built it, where it fits,
and where it goes next

Use cases, implementation status and roadmap
Companion to the B.Tech project report

Nabin Agrawal [Roll No.]
Kumar Vanshaj [Roll No.]
Parlin Kant Goyal [Roll No.]

Under the supervision of
[Supervisor Name]

Department of Computer Science and Engineering
International Institute of Information Technology, Bhubaneswar

November 5, 2026

Abstract

Dhuno is a web platform in which many people hear the same song at the same moment, each through their own device. The main project report explains how it is built. This companion document explains why it was built and where it is going. Our central idea is that the phones people already carry can replace much of the sound hardware needed at a concert, on a car trip or at a small party, and that personal playback also keeps sound close to the listener instead of spreading it into the surroundings. We map these scenarios to the current implementation (server-authoritative rooms, host or DJ control, location-based discovery, offline downloads), state plainly what is finished and what is not, and set out the work needed to make the scenarios dependable: clock-offset compensation, an offline nearby mode that works without internet, scaling for large audiences, and a measured evaluation.

Contents

1 Introduction **2**

2 Why we thought of Dhuno **2**

3 Use cases **3**

4 How the current design supports these scenarios **5**

5 What is implemented so far **6**

6 Where the system falls short of the use cases **7**

7 Future work **8**

8 Evaluation plan **10**

9 Roadmap **11**

10 Conclusion **11**

1 Introduction

The main report [10] (*Dhuno: A Real-Time Synchronized Music Streaming Platform with Host-Controlled Listening Rooms*) documents the problem statement, literature, design and implementation. It frames the project mostly around people who are in different places and want to hear the same song together.

This document widens that framing. We also want Dhuno to work when people are in the *same* place: a crowd at a concert, friends in a car, a room party with too few speakers. These settings change what the system must guarantee, so they matter for the design and for the future work. The document is organised as follows.

- Section 2 explains why we thought of the project.
- Section 3 describes the target use cases.
- Section 4 shows how the current design supports them.
- Section 5 lists what is implemented so far.
- Section 6 states honestly where the system falls short of the use cases.
- Sections 7–9 describe future work, how we will measure it, and the order we plan to do it in.

2 Why we thought of Dhuno

2.1 Listening became solitary, but enjoying music is still social

Streaming made music instantly available, but it also made listening a private activity. Sharing a song in person still means passing one earbud or pointing a phone at a friend. Sharing at a distance means a video call with a phone held near a speaker.

2.2 Shared listening needs hardware that is often missing

Playing music for a group normally depends on one device doing all the work.

- At a small party, in a hostel room or on a trip, there is often **no speaker at all**, or one small speaker that is too quiet for the whole group.
- A single speaker is a **single point of failure**: if its battery dies or the Bluetooth link drops, the music stops for everyone.
- A concert needs a large sound system, which is costly to hire, transport and set up.

2.3 Sound spreads to people who did not ask for it

A loud shared speaker is heard by everyone nearby, including neighbours, people studying or sleeping, and passers-by. This is ordinary noise pollution. If each listener hears the music through their own earphones or at their own volume, the sound stays near the listener and the surroundings stay quiet. Venues sometimes use this idea for “silent” events, but it normally needs dedicated transmitters and headsets.

2.4 The devices are already there

Everyone in the room already carries a phone. That phone is a speaker, an audio output for earphones, a screen for controls, and a network node. Our idea is to treat the crowd's devices as the sound system: one person (or a performer) is the *host*, and every other device plays the same track at the same moment.

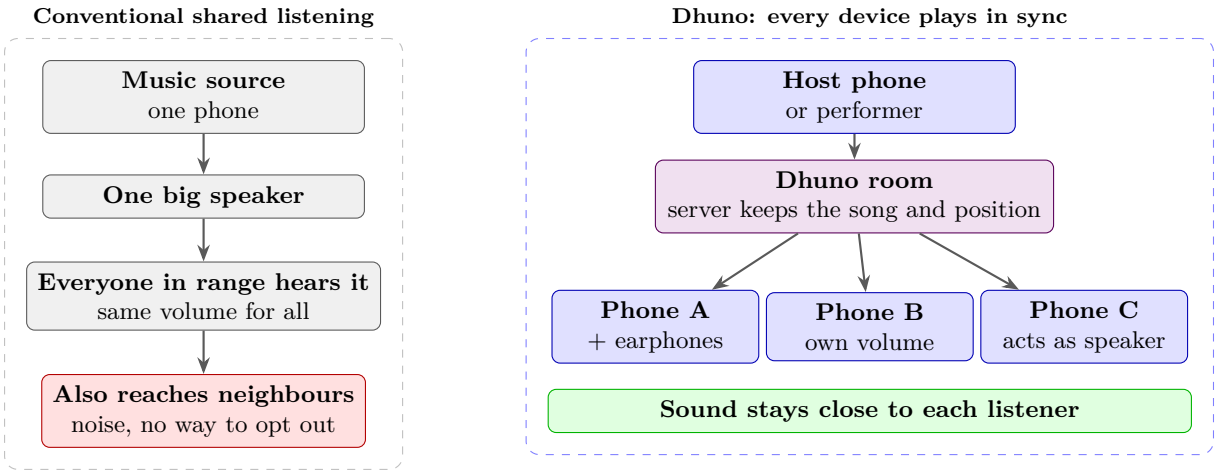


Figure 1: The core idea: replace one shared sound source with many synchronized personal ones.

2.5 Gaps in existing products

The main report reviews Spotify Jam [9], Apple SharePlay and the discontinued Turntable.fm and finds five gaps: ecosystem lock-in, weak host authority, no local discovery, fragile room state, and separated listener and creator tools. Dhuno answers these in a browser, with a host who controls the room and an artist area in the same app. None of those products is designed around the in-person scenarios below.

3 Use cases

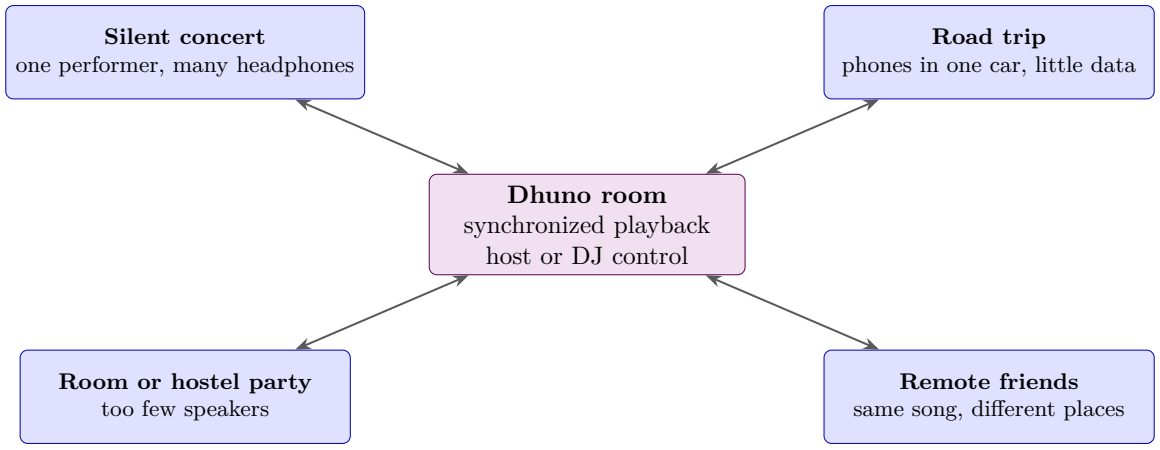


Figure 2: One room model, four settings.

3.1 UC1: Silent concert

A performer or DJ is the host. The audience wears headphones and joins the room on their phones, so everyone hears the same set while the venue itself stays quiet. This suits places where loud amplified sound is not allowed or not wanted, such as a campus lawn at night, a library event or a residential area. Dhuno's DJ mode already lets the host hand control to a chosen DJ. The audience can chat and react in the room. Today the set must come from tracks uploaded to Dhuno, not from live instruments.

3.2 UC2: Car or road trip

Passengers in a small car often have no speaker that suits everyone, and mobile data is unreliable on highways. The driver or a passenger hosts; others follow with earphones, or the phones themselves act as speakers. This case depends on songs being downloaded beforehand and on the group being able to connect without internet.

3.3 UC3: Room or hostel party

Ten people, one weak speaker. Several phones placed around the room can all play the same song, which together act as a distributed speaker system. If people want to stay quiet for neighbours, they can use earphones instead. The host controls the queue, and guests add songs and vote to skip.

3.4 UC4: Remote friends

The original scenario: friends in different places join one room over the internet. This is what the current release serves best.

Table 1: Scenarios compared. "Sync need" is how closely devices must agree.

Scenario	Who and where	Network	Sync need
UC1 Silent concert	One performer, many listeners with headphones, outdoors or in a hall	Crowded mobile or venue Wi-Fi	Low between listeners (each hears only their own headphones). Higher against a visible performer.
UC2 Road trip	2 to 5 people in one car	Often no data	High if phones act as speakers, low with earphones.
UC3 Room party	5 to 20 people, several phones as speakers	Home Wi-Fi, or none	High: a few tens of milliseconds of difference between nearby speakers is heard as echo.
UC4 Remote friends	Any number, anywhere	Internet	Moderate: each listener hears only their own device.

The table shows the important pattern. **When listeners are close together and their devices act as speakers, synchronization must be much tighter than for remote friends, and the system should not depend on the internet.** The main report explicitly targets "perceptually acceptable synchronization for casual group listening". The in-person scenarios push beyond that target, which drives the future work in Section 7.

4 How the current design supports these scenarios

4.1 Architecture

Dhuno is a three-tier web application with a separate real-time channel (Figure 3). The browser runs a React single-page app. An Express server handles REST calls, and a Socket.IO gateway [4], built on WebSockets [1], on the same HTTP server keeps live rooms in sync. MongoDB stores durable data, Redis [6] holds fast-changing state, and Cloudinary stores audio and images.

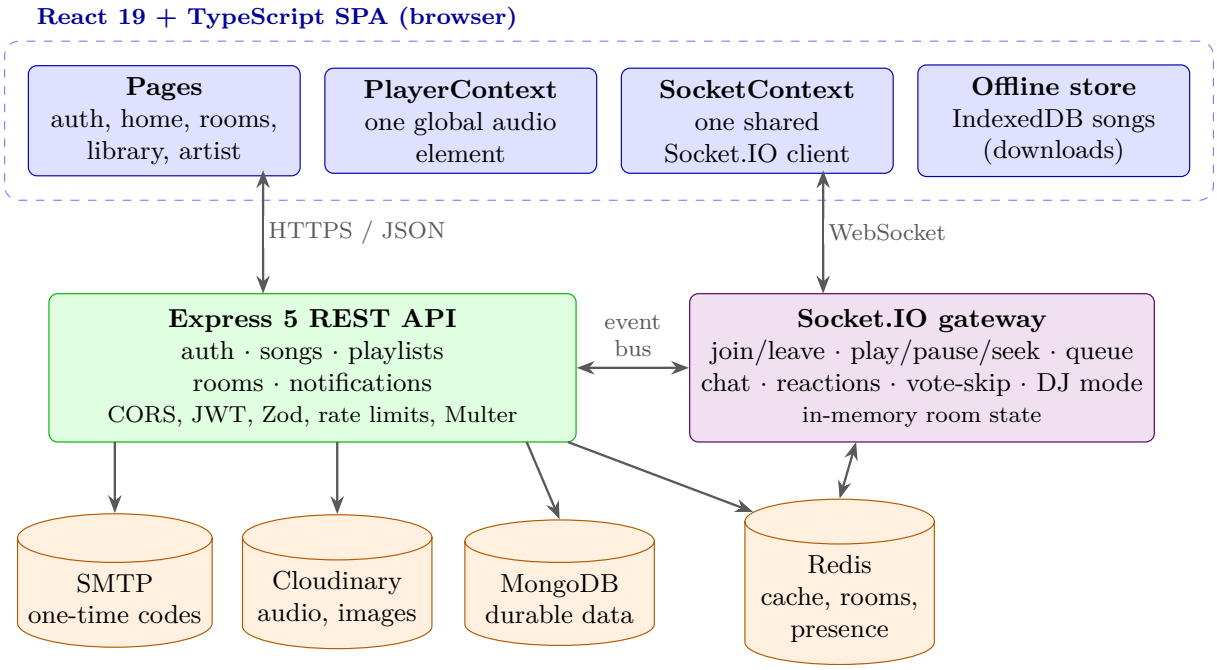


Figure 3: Implemented system architecture. Sockets also read MongoDB to rebuild a room that is not in memory or Redis.

4.2 Why this fits shared listening

- **One reference instant.** The server does not stream the position. When the host plays or seeks, the server stores the instant at which the song's position would have been zero:

$$T_{start} = T_{now} - 1000\, p_0, \tag{1}$$

  where p_0 is the starting position in seconds and times are in milliseconds. Every listener then computes its own position:

$$p(t) = \frac{t - T_{start}}{1000}. \tag{2}$$

  Network traffic is proportional to host actions, not to elapsed time.

- **Host-only control.** The check "may this user control playback?" runs on the server for every control event, so a modified client cannot take over the room. A host can delegate control to a DJ.

- **Late join and reconnection.** A newcomer, or a phone that lost its connection, receives the whole room state and seeks to $p(t)$ at once. State is snapshotted to Redis, so a room survives a server restart.

- **Nearby discovery.** Rooms store a GeoJSON point with a `2dsphere` index [5], so public rooms within one kilometre of a user are found with one query.

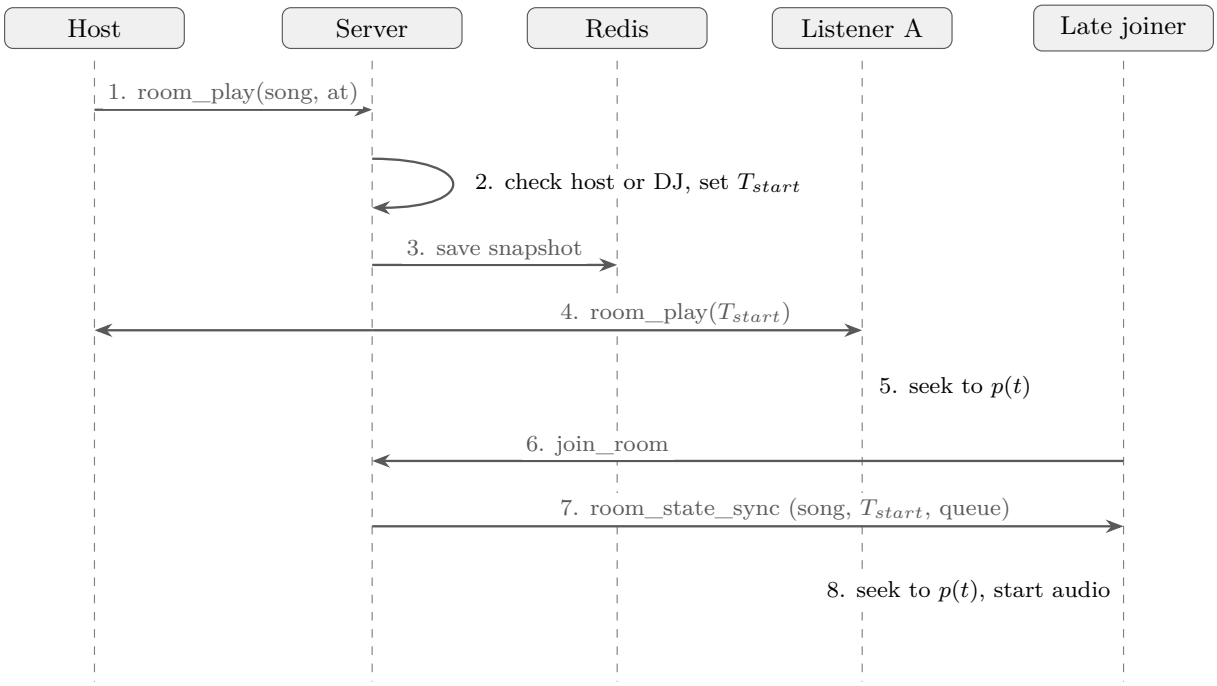


Figure 4: Message flow for a host play action (steps 1 to 5) and a late join (steps 6 to 8).

4.3 How the scenarios map onto existing parts

Table 2: What each scenario needs and which part of Dhuno provides it.

Need	Dhuno component	Status
Everyone on the same second	Reference instant T_{start} and client-side seek	**Partial** (assumes clocks agree)
Only the performer or host controls playback	Server-side check, DJ mode	**Done**
Latecomer or dropped phone catches up	`room_state_sync`, Redis snapshot	**Done**
Find a room nearby	GeoJSON and `2dsphere` index, 1 km radius	**Done**
Performer's own tracks	Artist upload to Cloudinary	**Done**
Music without internet	IndexedDB downloads	**Partial**
Connect nearby devices without internet	Nearby mode (Section 7.2)	**Planned**

5 What is implemented so far

The system is a working full-stack application, written in TypeScript on both sides. The frontend uses React 19, Vite, Tailwind CSS 4 and React Router 7. The backend uses Node.js, Express 5, Socket.IO 4 and Mongoose, with Redis through ioredis and Cloudinary for media. Table 3 lists the features and their status.

Table 3: Implementation status.

Area	Details	Status
Accounts and sessions	Sign-up, login, JWT in an httpOnly cookie or bearer header, bcrypt hashing, profile editing	**Done**
Two-factor and recovery	E-mailed one-time codes for 2FA and password reset, backup codes, per-route rate limits	**Done**
Google sign-in	Works, but the server trusts the profile sent by the client instead of verifying Google's ID token	**Partial**
Player and library	One global audio element, playlists with drag-and-drop ordering, liked songs, recently played	**Done**
Search and trending	Case-insensitive title and artist search (20 results), trending list cached for 5 minutes	**Done**
Rooms	Create, join, leave, public or private, location filter within 1 km	**Done**
Control and roles	Host or DJ only playback, DJ mode and assignment, host promotion when the host leaves	**Done**
Resilience	Full state sync on join, Redis room snapshot (1 h), presence heartbeat (90 s), restore after restart	**Done**
Room social features	Chat, reactions, shared queue with reorder and remove, vote to skip	**Done**
Artist workspace	Upload songs and cover art (25 MB limit, MIME whitelist), artist playlists, dashboard	**Done**
Artist clips and analytics	Clip pages and analytics pages exist, depth of features still growing	**Partial**
Themes and notifications	Theme picker, in-app notifications; mood-aware themes not started	**Partial**
Offline downloads	Songs saved as blobs in IndexedDB and played from there. The app shell is not cached, so the page itself still needs a network to open	**Partial**
Deployment	Docker images for both apps, Nginx static hosting, GitHub Actions deploying the backend to Azure Web App	**Done**
Clock-offset compensation	Not implemented, see Section 6.1	**Planned**
Automated tests and load tests	Scenarios are checked by hand; no automated suite yet	**Planned**
Background job queue	BullMQ is a dependency but unused; e-mail is sent inline	**Planned**

6 Where the system falls short of the use cases

6.1 The most important gap: synchronization

The server computes T_{start} from *its own* clock, but each listener's browser evaluates $p(t)$ with the *local* clock (`Date.now()`). The formulas therefore assume that server and client clocks agree. A phone whose clock is two seconds off will play two seconds off, with no way to notice. Phone clocks are usually corrected by the network, which is why the approach works for casual use, but that is not guaranteed, and the in-person scenarios depend on it much more.

For headphone listening (UC1, UC4) some drift is tolerable. For several phones acting as speakers in one room (UC3), a difference of a few tens of milliseconds is heard as echo. The report's own scope statement excludes sample-accurate alignment, and we agree it is not needed, but it does need to be *better than now and measured*.

Table 4: Gaps between the use cases and the current implementation.

Gap	Why it matters	Planned fix
G1 Clock offset not compensated	Listeners drift apart, phones used as speakers produce echo	NTP-style handshake and drift threshold
G2 Device output latency differs	Bluetooth headphones add delay, so the same instant sounds different per device	Per-device calibration
G3 Needs internet and a server	Cars and crowded venues have poor connectivity	Offline nearby mode
G4 One server instance, in-memory state	A large audience cannot be served by one process	Socket.IO Redis adapter, several instances, load tests
G5 Only stored tracks	A live band cannot be streamed; a DJ set from uploaded tracks can	Live audio ingest, later
G6 App shell not cached	The app cannot open offline even if songs are downloaded	Service worker
G7 Google token not verified server-side	A forged profile could claim an account	Verify the ID token on the server
G8 No measurements	Claims about sync quality cannot yet be proven	Evaluation plan (Section 8)

7 Future work

7.1 Clock-offset and latency compensation

Each client estimates the offset θ between its clock and the server clock with four timestamps, as in the Network Time Protocol [2]: its request time t_0, the server's receive and send times t_1 and t_2, and its own receive time t_3:

$$\theta = \frac{(t_1 - t_0) + (t_2 - t_3)}{2}. \tag{3}$$

It repeats the exchange several times and keeps the estimate from the shortest round trip, which is the least disturbed by queuing. The position then becomes

$$p(t) = \frac{t + \theta - T_{start}}{1000} + \lambda, \tag{4}$$

where λ is a per-device correction for audio output delay, for example when Bluetooth headphones are used. The client seeks only when $|p(t) - \text{currentTime}| > \delta$, for a threshold δ tuned by experiment, so tiny differences do not cause audible jumps. Cristian's algorithm [3] is a simpler variant that we can use as a baseline.

7.2 Offline nearby mode on the web

For the car and room scenarios, devices need to sync with no internet. Dhuno is a website, and a browser cannot do some of the obvious things. Web Bluetooth only lets a page connect *out* to a device; a page cannot advertise a room or accept connections, and iOS Safari does not support it. Browsers also have no API for Wi-Fi Direct or for scanning nearby devices. A native app wrapper would remove these limits, but it changes the product from a website to an app, so we treat it as a later option.

The web-only design uses a local network that needs no internet (a phone hotspot or a router) and WebRTC data channels [7, 8] between the devices (Figure 5).

1. **Offline app shell.** A service worker caches the app so it opens with no network. The IndexedDB song store already exists.
2. **Local network.** All devices join the same hotspot or router. No internet is required.
3. **QR-code pairing.** Without a signalling server, devices exchange the WebRTC connection offer and answer by scanning each other's QR codes. After that, the browsers talk directly.
4. **Local room engine.** The host's device runs the same room logic as the Socket.IO gateway (same events: play, pause, seek, queue, join) and sends them over the data channels. A `Transport` interface lets the UI use either Socket.IO online or data channels offline.
5. **File sharing.** Songs a guest lacks are sent from the host in chunks over the data channel and stored in IndexedDB before playback.
6. **Clock sync.** The ping-pong exchange of Section 6.1 runs over the data channel, so offline playback is scheduled as "play X at time T".

Audio is never streamed live over the link. Every device plays its own local copy, started at the agreed time.

Limits we expect. Pairing is manual, so there is no automatic list of nearby rooms. Some phones and routers block device-to-device traffic (client isolation), which we must test on real hotspots. Browsers may pause audio or connections when the screen locks, especially on iOS. Each guest is a separate peer connection to the host, so the group size is small; we expect a handful of guests per host and will measure the real limit.

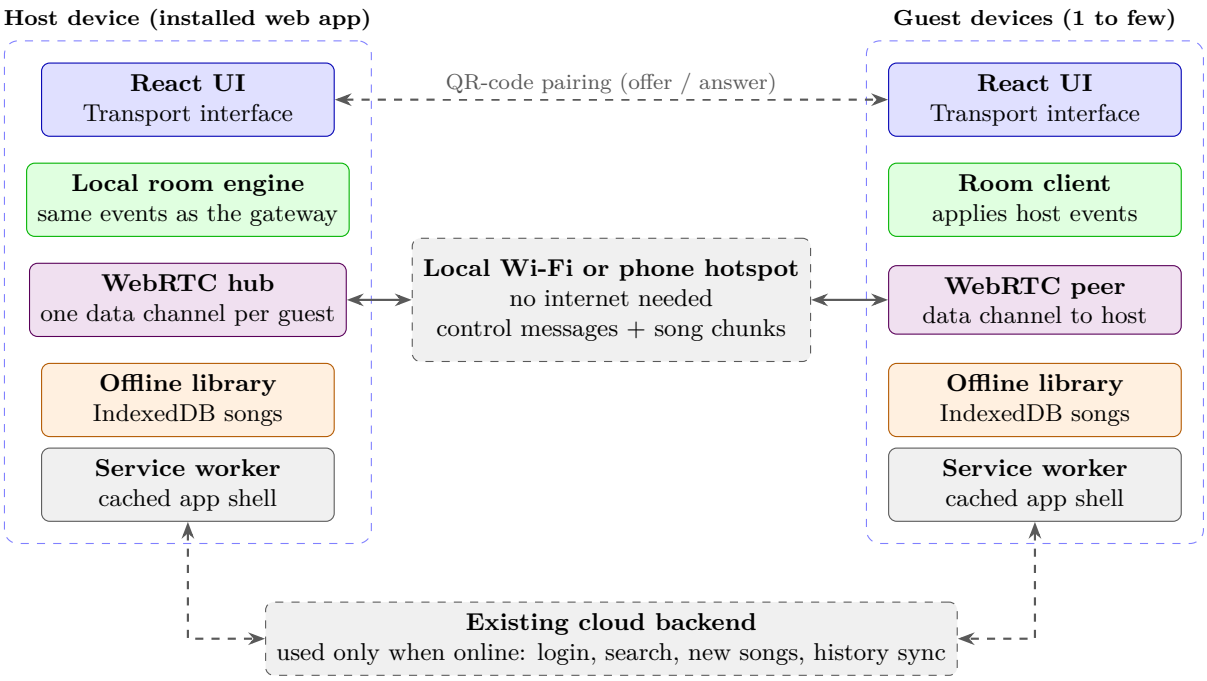


Figure 5: Proposed offline nearby mode for the web: PWA, local network, WebRTC data channels and QR-code pairing.

Native option. If the project later becomes an installed app (for example by wrapping the web app with Capacitor), Bluetooth LE can be used for automatic discovery and small control messages, and Wi-Fi Direct or Nearby Connections for copying song files. Bluetooth LE is too slow to carry audio, so the same rule applies: devices share files first, then play locally on a shared clock.

7.3 Concert mode

A silent concert has one host and a large audience, so the single-instance design must grow (Figure 6). The plan is to use the Socket.IO Redis adapter so that several gateway instances share rooms through Redis, to place a load balancer in front, and to run load tests that measure the number of listeners per instance before latency grows. Other items are a larger audience view for the host, a reaction wall, and, later, live audio ingest so a performer can be heard live and not only from uploaded tracks.

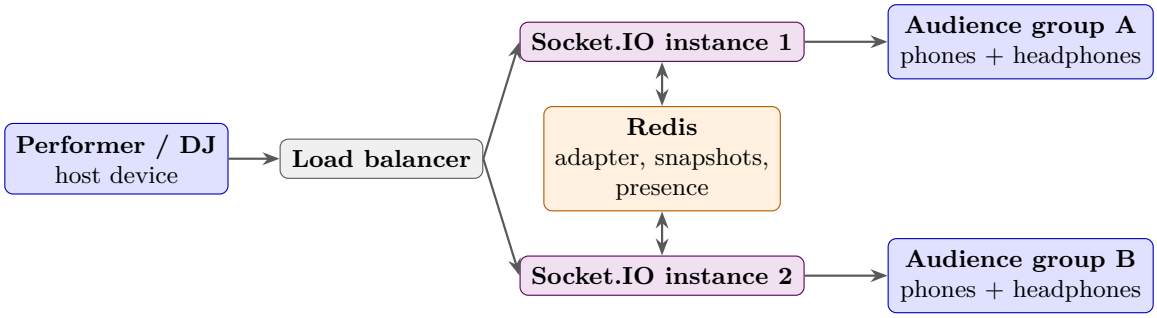


Figure 6: Proposed scaling for a large audience. Rooms are shared across instances through Redis.

7.4 Speaker-array mode

For UC3, several phones in one room can play the same track as a distributed speaker system. Beyond tight synchronization, an extra idea is to let the host give each phone a role (left, right, or bass-heavy) so the room sounds fuller. This depends on the sync work in Section 6.1 and is the most experimental item here.

7.5 Other planned work

These items come from the main report and remain on the list.

- **Security.** Verify Google ID tokens on the server, replace the wildcard cross-origin header on the static host with the known frontend origin, and review the large request-body limit.
- **Search.** Indexed, typo-tolerant, ranked search with escaped input and cached popular queries.
- **Recommendations.** Popularity and content-based suggestions from plays, likes and room activity, then collaborative filtering and recommended rooms.
- **Background jobs.** Move e-mail and clean-up into BullMQ workers.
- **Testing.** Automated unit and integration tests, end-to-end tests with several simulated clients, and load tests.
- **Interaction.** Voice commands, mood-aware themes, invitation links.

8 Evaluation plan

The main report claims no latency figures because none were measured. The use cases make measurement necessary. Table 5 lists what we will measure. The targets are hypotheses to be tuned from the first results, not promises.

Table 5: Planned measurements.

Metric	How we measure it	Target
Start-time spread between listeners	Play a click track on several phones and record all of them with a separate recorder. Compare the click onsets. Also log each client's expected versus actual position.	Tens of milliseconds or better, compared before and after clock compensation
Late-join convergence	Time from `join_room` to audio at the correct position.	Under a few seconds
Reconnection recovery	Drop and restore a client's connection, and restart the server, then check the room state.	State restored with no manual step
Capacity per instance	Simulated Socket.IO clients joining one room until event latency grows.	Find the limit, then raise it with the Redis adapter
Offline pairing and transfer	QR pairing success rate, and transfer time per song on a hotspot and on a router.	Reliable on at least two phone models

9 Roadmap

We propose doing the work in the order that unlocks the use cases soonest. Synchronization accuracy and measurement come first, because every in-person scenario depends on them.

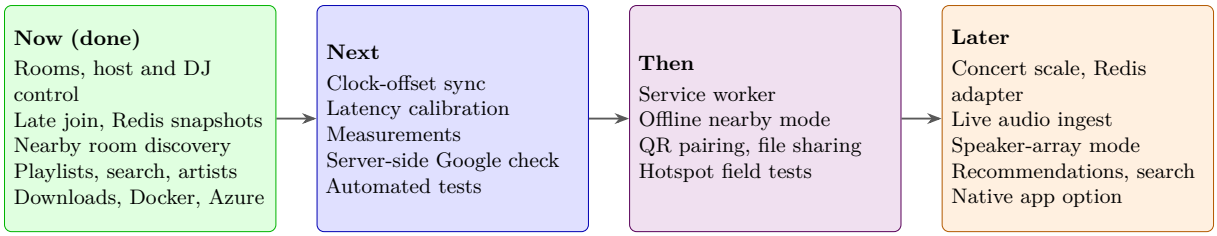


Figure 7: Proposed order of work.

10 Conclusion

Dhuno started from a simple observation: people want to hear music together, and the hardware and noise that usually come with that are a poor fit for concerts, cars and small parties. A synchronized room model, with a host who controls playback and every phone playing its own copy, addresses all three. The current release already delivers the room model, host and DJ control, late join, nearby discovery, an artist workspace and offline downloads, and it serves remote friends well.

The in-person scenarios ask for more than the release gives: tighter, measured synchronization, the ability to work without internet, and capacity for large audiences. We have identified these gaps, designed a web-only way to close them, and ordered the work so that measurement comes first. Completing it would turn Dhuno from a way to listen together at a distance into a way to share a sound system among everyone in the room.

References

[1] I. Fette and A. Melnikov, "The WebSocket Protocol," RFC 6455, IETF, Dec. 2011.

[2] D. Mills, J. Martin, J. Burbank and W. Kasch, "Network Time Protocol Version 4: Protocol and Algorithms Specification," RFC 5905, IETF, Jun. 2010.

[3] F. Cristian, "Probabilistic clock synchronization," *Distributed Computing*, vol. 3, no. 3, pp. 146–158, 1989.

[4] Socket.IO Documentation, v4. [Online]. Available: https://socket.io/docs/v4/

[5] MongoDB Inc., "2dsphere indexes," MongoDB Manual. [Online]. Available: https://www.mongodb.com/docs/manual/core/2dsphere/

[6] Redis Documentation. [Online]. Available: https://redis.io/docs/

[7] W3C, "WebRTC 1.0: Real-Time Communication Between Browsers," W3C Recommendation, Jan. 2021.

[8] R. Jesup, S. Loreto and M. Tuexen, "WebRTC Data Channels," RFC 8831, IETF, Jan. 2021.

[9] "Spotify launches Jam, a real-time collaborative playlist controlled by up to 32 people," *TechCrunch*, Sep. 26, 2023.

[10] N. Agrawal, K. Vanshaj and P. K. Goyal, "Dhuno: A Real-Time Synchronized Music Streaming Platform with Host-Controlled Listening Rooms," B.Tech project report, IIIT Bhubaneswar, Oct. 2026.